

# Leva.Framework.Engine

`Leva.Framework.Engine` is the runtime implementation of ArchFramework. It executes the event-driven state-machine model defined by Core.

Engine turns Core contracts into deterministic runtime behavior. It owns event queueing, dispatching, transition draining, state execution, routine execution, fallback behavior execution, alarms, statuses, runtime history, logging, and snapshot creation/loading. `Leva.Framework.Engine` depends on `Leva.Framework.Core`. Engine must not depend on Fakes, Storage implementations, UI providers, notification providers, authentication providers, database providers, devices, or application projects.

```
Leva.Framework.Engine
    -> Leva.Framework.Core
```

## Project overview

Engine makes application behavior run through one controlled path. Events enter `EventQueue`, `EventLoop` dequeues them, `EventDispatcher` sends them through runtime hooks, the active state handles them, an active routine may continue, fallback behaviors may run, and queued transitions are applied only after event handling finishes.

```
EventQueue
    -> EventLoop
    -> EventDispatcher
    -> IStatusUpdater
    -> IAlarmSupervisor
    -> StateMachine
    -> RoutineRunner
    -> BehaviorRunner
    -> QueuedTransition drain
```

This separation keeps state changes predictable. Runtime code requests transitions through `ITransition`, while `StateMachine` controls when those transitions are applied. Engine also keeps latest-known runtime facts in boards and chronological history in `RuntimeLog`.

```
AlarmBoard    -> active alarms
StatusBoard   -> latest statuses
RuntimeLog    -> chronological LogEntry history
LogSink       -> diagnostic output
Context       -> runtime composition root
```

The engine stays application-independent. Applications provide concrete states, events, access objects, services, screens, persistence, notifications, and domain rules. Engine provides reusable runtime mechanics.

Engine intentionally does not depend on `Leva.Framework.Execution`. Long-running or external work should be started through Execution by the application composition layer, then reported back to Engine through ordinary events when the application wants workflow state to react.

## Files and classes

---

### Composition and runtime execution

`Context` - Main runtime composition object that owns engine services and exposes controlled runtime capabilities to the host. `ContextBuilder` - Builds a `Context` from clocks, log sinks, queues, supervisors, updaters, state bindings, and behavior bindings.

### Events and dispatching

`IEventQueue` - Queue contract used by the event loop. `EventQueue` - Priority-aware queue for immediate and delayed events. `QueuedEvent` - Event plus priority and enqueue time. `EventLoop` - Runs queued events through the dispatcher until cancelled. `EventDispatcher` - Routes one event through status update, alarm supervision, state handling, routine handling, behavior fallback, and transition draining.

### State machine and transitions

`StateMachine` - Owns the current state, calls enter/exit lifecycle methods, routes events to the current state, and applies queued transitions. `StateBinding<TAccess>` - Internal binding from a state to the access factory that creates its typed capability surface. `QueuedTransition` - Internal transition implementation that captures `To` and `Reenter` requests until the state machine drains them.

### Routines and behaviors

`RoutineRunner` - Starts, cancels, and routes events to the currently active routine. `RoutineBinding<TAccess>` - Internal binding from a routine to its typed access factory. `BehaviorRunner` - Runs fallback behaviors when an event is not handled by alarm, state, or routine logic. `BehaviorBinding<TAccess>` - Internal binding from a behavior to its typed access factory.

### Runtime boards and hooks

`AlarmBoard` - Stores active alarms and exposes safe snapshots of alarm entries. `IAAlarmSupervisor` - Hook that can map events to alarm behavior and optionally handle events before state logic. `NullAlarmSupervisor` - Default alarm supervisor that intentionally handles nothing. `StatusBoard` - Stores latest-known status values and exposes safe snapshots. `IStatusUpdater` - Hook that can update status

memory from incoming events. `NullStatusUpdater` - Default status updater that intentionally updates nothing.

## Runtime logging

`RuntimeLog` - Stores chronological runtime history and mirrors entries to an `ILogSink`. `LogEntry` - Core value representing one structured runtime log entry. `LogCategory` - Core value that groups runtime log entries by broad framework area. `MemoryLogSink` - Thread-safe in-memory log sink for diagnostics and tests. `NullLogSink` - Log sink implementation that intentionally ignores log entries.

## Snapshot support

`Context.CreateSnapshot` - Captures current state ID, active alarms, statuses, and optional host data into a `Snapshot`. `Context.LoadSnapshotAsync` - Restores alarms, statuses, and state-machine position from a `Snapshot`.